AgentOS: Architectural Overview & Discussion

Welcome to the deep dive on **AgentOS (AOS)**. This document breaks down the theory, design, and execution flow of the Proof-of-Concept we just built.

1. The Problem Solved

Large Language Models (LLMs) are incredibly powerful reasoning engines, but they are fundamentally **stateless text generators**. Out of the box, they cannot:

1. Maintain long-term memory (due to context window limits).
2. Execute code safely.
3. Run as continuous, manageable background processes.

AgentOS solves this by borrowing concepts from traditional Operating Systems (like Linux or Windows). Instead of scheduling CPU time for a binary executable, AgentOS schedules **Context Windows** for an **LLM**, acting as the "Kernel" that bridges the AI to the real world securely.

---

2. Background Knowledge & Concepts

To understand AgentOS, we map traditional OS concepts directly to AI concepts:

Traditional OS Concept	AgentOS Equivalent	Description
Process Control Block (PCB)	AgentPCB	Tracks the agent's ID (PID), status (RUNNING, SLEEPING), token count, and active context.
CPU Scheduler	**Go Orchestrator**	Manages the infinite loop of prompting the LLM and handling its outputs.
RAM (Active Memory)	**LLM Context Window**	The immediate conversation history fed to the model.
Swap File / Hard Drive	**ChromaDB (Vector DB)**	When the context window gets too full, older messages are "paged out" to ChromaDB and retrieved via semantic search when needed.
System Calls (Syscalls)	[SYS_CALL::NAME]	Standardized JSON commands the LLM outputs to request the Kernel to do something real (e.g., read a file, run a bash command).

Traditional OS Concept	AgentOS Equivalent	Description
Hardware Drivers	**Go pkg/drivers**	Go code that safely translates the LLM's Syscall into actual cloud infrastructure actions (Docker, R2, ChromaDB).

---

3. Core Architecture

1. The Go Kernel (Orchestrator)

The core of AgentOS is written in Go (cmd/aos/main.go). It is a highly concurrent, low-memory engine. When you run aos spawn, the Kernel creates a new AgentPCB (Process Control Block) and spins up a dedicated lightweight Go thread (Goroutine) to manage that specific AI agent's lifecycle.

---

4. Execution Flow (The Agent Loop)

Here is the exact lifecycle of a spawned agent, visualized:

5. Design Decisions for this POC

TIP

Why Go instead of Python? Python is standard for AI, but it is heavy and struggles with massive concurrency due to the GIL. Go's goroutines allow AgentOS to manage thousands of sleeping/blocked AI agents with mere megabytes of RAM overhead.

IMPORTANT

Why Docker instead of E2B for the POC? E2B is fantastic for cloud deployments. However, integrating the official Docker Engine API directly into our Go backend gives us immediate, zero-configuration sandboxing locally without requiring you to set up external E2B API keys during the initial testing phase. It perfectly mimics the required "Isolated Container" spec while remaining completely self-contained.